RAPPORT FINAL - PROJET DE ROBOTIQUE LU2IN013

Sommaire

1. **Introduction**
2. **Présentation du Projet**
3. **Organisation du Travail**
4. **Description Technique Détaillée**
5. **Défis Rencontrés et Solutions**
6. **Résultats Obtenus**
7. **Conclusion et Perspectives**
8. **Annexes**

1. Introduction

Ce projet de robotique a été réalisé dans le cadre du module LU2IN013. Durant plusieurs semaines, notre équipe a développé une solution complète permettant à la fois la simulation virtuelle d'un robot mobile et le contrôle d'un robot physique GoPiGo3. Ce rapport présente notre travail, depuis les choix techniques initiaux jusqu'aux résultats finaux, en passant par l'organisation de l'équipe et les défis rencontrés.

2. Présentation du Projet

2.1 Objectifs Principaux

L'objectif central était de créer une plateforme permettant :

- La **simulation précise** des déplacements d'un robot dans un environnement 2D

- Le **contrôle en temps réel** d'un robot GoPiGo3
- L'implémentation de **stratégies de mouvement**
- La **détection et évitement d'obstacles**

2.2 Choix Technologiques

Notre solution s'appuie sur :

- **Tkinter** pour l'interface graphique
- **Architecture MVC** pour une séparation claire des responsabilités
- **GoPiGo3 API** pour le contrôle du robot physique
- **Git** pour la gestion de versions

3. Organisation du Travail

3.1 Méthodologie Agile

Nous avons adopté une approche Scrum avec :

- **Sprints de 1 semaine** avec objectifs clairs
- **Réunions** pour suivre l'avancement
- **Tableaux Trello** organisés en :
    - Backlog
    - En cours
    - En test
    - Terminé

3.2 Répartition des Rôles

Chaque membre du groupe prend une ou plusieurs taches selon les disponibilités de chaque personne.

3.3 Outils Collaboratifs

- **GitHub** : 3 dépôts (main, test, version-6-avril)
- **WhatsApp**
- **Trello** : 12 cartes en moyenne par sprint

4. Description Technique Détaillée

4.1 Architecture Globale

Notre solution suit une architecture MVC :

Modèle :

- `Robot.py` : Gère la physique et la position
- `Simulation.py :` Orchestre la cohérence globale entre le robot et son environnement en gérant la boucle principale de simulation et les collisions
- `Environnement.py` : Crée et gère les obstacles

Vue :

- `AffichageGraphique.py` : Interface Tkinter

Contrôleur :

- `Strategie.py` : Comportements programmables
- `Controleur.py` : Orchestration des stratégies

4.2 Modules Clés

1. Fichiers Principaux

1. **main_graphique.py**

   Ce fichier constitue le cœur de la simulation graphique. Il initialise l'interface utilisateur en créant une fenêtre Tkinter qui servira de cadre à notre environnement virtuel. Après avoir positionné notre robot au centre de l'espace de travail, il met en place le système d'adaptation pour la simulation et instancie le contrôleur principal. La particularité de ce module réside dans sa capacité à lancer des stratégies de déplacement complexes, comme le parcours en carré, tout en gérant de manière optimale les ressources système grâce à un thread dédié pour les calculs de simulation.

2. **main_reel.py**

   Dédié au contrôle du robot physique, ce composant établit la communication avec le matériel GoPiGo3 via la bibliothèque officielle. Il configure précisément les paramètres mécaniques du robot (dimensions des roues, entraxe) avant d'exécuter les stratégies programmées. Contrairement à la version simulée, ce module doit gérer les contraintes du monde réel, comme la latence des commandes ou les imprécisions des encodeurs.

2. Gestion de l'Environnement

3. **environnement.py**

   Ce module intelligent crée dynamiquement un espace de travail peuplé d'obstacles positionnés aléatoirement, tout en respectant une marge de sécurité pour éviter les configurations impossibles. Sa fonction de détection de collision repose sur un calcul de distance précis entre le robot et chaque obstacle, permettant une réaction immédiate en cas de risque de collision.
   L'environnement ainsi généré offre un cadre réaliste pour tester les capacités d'évitement du robot.

3. Noyau de Contrôle

4. **controleur.py**

   Véritable chef d'orchestre du système, cette classe gère avec précision l'exécution des différentes stratégies. Son mécanisme de file d'attente permet d'enchaîner automatiquement les actions programmées, tandis que sa boucle de contrôle principale veille au bon déroulement des opérations. La gestion fine de l'état d'exécution (via la variable 'running') assure une interruption propre des processus lorsque nécessaire.

4. Stratégies de Déplacement

5. **strategie.py**

   Ce fichier regroupe l'ensemble des comportements programmables du robot. Chaque stratégie (avance, rotation, recul) est encapsulée dans une classe dédiée, suivant un modèle uniforme avec ses méthodes start(), step() et stop(). La stratégie séquentielle permet de composer ces actions élémentaires en parcours complexes, comme le fameux carré qui sert de référence pour valider le bon fonctionnement du système. Les stratégies conditionnelles introduisent quant à elles une logique décisionnelle basée sur l'environnement.

5. Simulation Physique

6. **simulation.py**

   Ce module fait le lien entre le robot et son environnement. Sa méthode updateModele(), appelée régulièrement, assure la cohérence physique du système en vérifiant notamment les collisions. La simulation peut être lancée pour une durée déterminée, avec interruption automatique en cas de détection d'obstacle.

6. Modélisation du Robot

7. **robot.py**

   Cette classe matérialise le concept de robot mobile dans notre simulation. Elle gère avec précision la position, l'orientation et les caractéristiques physiques du robot. Les équations cinématiques implémentées dans la méthode move() traduisent les vitesses des roues en mouvement réaliste, tandis que le système de détection d'arrêt permet une réaction immédiate aux commandes.

7. Adaptateurs Matériels

8. **adaptateur_robot_reel.py**

   Interface cruciale avec le monde physique, cet adaptateur traduit les commandes de haut niveau en signaux compréhensibles par le GoPiGo3. Il gère particulièrement bien l'estimation de la position via les encodeurs, tout en offrant une abstraction des spécificités matérielles.

9. **adaptateur_robot_simule.py**

   Version virtuelle de l'adaptateur, il émule le comportement du robot réel en calculant la position théorique basée sur les vitesses des roues. Son rôle est essentiel pour le développement et les tests sans accès au matériel.

8. Interface Graphique

10. **affichage_graphique.py**

    Ce module transforme les données abstraites en une représentation visuelle claire et intuitive. Son canvas Tkinter affiche en temps réel le robot (symbolisé par un cercle bleu), sa trajectoire (ligne grise) et les obstacles (cercles rouges). La mise à jour périodique à 20 images par seconde assure une visualisation fluide, tandis que la flèche de direction permet de vérifier instantanément l'orientation du robot.

5. Défis Rencontrés et Solutions

5.1 Problème de Synchronisation

Symptôme : L'interface graphique gelait pendant les calculs

Solution : Implémentation d'un système multi-thread

5.2 Imprécision des Rotations

Symptôme : Erreur moyenne de 8° sur les rotations

Solution : marge d'erreur de 5°

5.3 Détection de Collisions

Symptôme : Faux positifs fréquents

Solution : Algorithme de distance optimisé

6. Résultats Obtenus

- Simulation 2D fluide
- Contrôle du GoPiGo3
- Stratégies de mouvement

7. Conclusion et Perspectives

7.1 Bilan du Projet

- Expérience précieuse en travail d'équipe
- Maîtrise de Python/Tkinter

7.2 Améliorations Possibles

1. Interface 3D
2. Algorithmes d'évitement avancés

7.3 Retour d'Expérience

Ce projet nous a permis d'appliquer concrètement les méthodologies agiles tout en développant des compétences techniques variées.

8. Annexes

8.1 Code Source

Dépôt GitHub : https://github.com/Projet-Robot-5/demo-repository/blob/main/robot_simulation

8.2 Bibliographie

- Documentation Tkinter
- Manuel GoPiGo3
- Cours LU2IN013

Fait par : Dana Cohen , Avi Skoury , Rima Baghdadi , Nada Ben Alaya , Wiem Smida

Date : 27/05/2025

Encadrant : Olivier Sigaud – Nicolas Baskiotis